

RAPPORT DE PROJET

Système de reconnaissance faciale en temps réel

Python - OpenCV - DeepFace - Jupyter Notebook

Module	Machine Learning
Encadré par	Pr. Khalid El-aoula
Réalisé par	Farah Boudouma et Mohamed Amine El Kadi
Filière	Informatique appliquée
Année universitaire	2025-2026
Environnement	Jupyter Notebook - Python

Ce projet a pour objectif de développer une application simple capable d'identifier une personne à partir d'une webcam. Les données utilisées sont des photos personnelles de deux membres du groupe, enregistrées avec leur accord dans un dossier local.

Projet académique - Données utilisées uniquement à des fins d'apprentissage

1. Introduction et problématique

La reconnaissance faciale est une application de la vision par ordinateur qui permet de détecter un visage puis de le comparer à des visages déjà connus. Dans notre projet, la caméra de l'ordinateur fournit des images en temps réel. Le programme compare ensuite le visage observé avec les photos enregistrées dans un dossier appelé **visages_connus**

Les données ne proviennent pas d'Internet ni d'une base publique. Elles ont été créées par les membres du groupe à partir de leurs propres visages. Chaque fichier porte le nom de la personne, par exemple « farah.jpg ». Ce nom est ensuite récupéré automatiquement par le programme lorsqu'une correspondance est trouvée.

Problématique

Comment créer, avec un code simple, une application capable de reconnaître en temps réel une personne connue à partir d'une petite base de photos personnelles ?

Objectifs du projet

- Ouvrir la webcam et lire les images en continu.
- Comparer le visage capturé avec les photos enregistrées.
- Afficher le nom de la personne reconnue ou le mot « Inconnu ».
- Limiter la reconnaissance à une fois toutes les deux secondes pour réduire le temps de calcul.
- Fermer correctement la caméra avec la touche Q.

Problème rencontré dans le projet

Le principal problème est la faible taille de la base de données. Avec une seule photo par personne, le système peut fonctionner lorsque la personne est bien éclairée et placée de face, mais il peut afficher « Inconnu » si l'angle, l'expression ou la lumière change. Le modèle doit donc généraliser à partir de très peu d'exemples, ce qui limite la fiabilité.

2. Outils et organisation des données

Python	Langage principal utilisé pour écrire le programme.
Jupyter Notebook	Environnement dans lequel le code est exécuté cellule par cellule.
OpenCV (cv2)	Ouverture de la webcam, lecture des images, affichage du texte et de la fenêtre vidéo.
DeepFace	Comparaison du visage capturé avec les visages enregistrés.
os et time	Gestion des noms de fichiers et exécution de la reconnaissance toutes les deux secondes.

Organisation de la base d'images

```
FaceRecognition.ipynb
visages_connus/
    farah.jpg
    autre_personne.jpg
```

Le dossier doit se trouver dans le même emplacement que le notebook. Le nom du fichier devient le nom affiché à l'écran. Ainsi, si DeepFace trouve une correspondance avec « farah.jpg », le programme enlève l'extension « .jpg » et affiche simplement « farah ».

Importation des bibliothèques

```
import cv2
import os
import time
from deepface import DeepFace

dossier = "visages_connus"
camera = cv2.VideoCapture(0)
nom = "Recherche..."
dernier_test = 0
```

La ligne VideoCapture(0) ouvre la caméra principale. La variable « nom » contient le texte affiché sur la vidéo, tandis que « dernier_test » sert à contrôler le délai entre deux recherches.

3. Fonctionnement du programme

1. Lecture de la caméra - La boucle while True lit une nouvelle image avec camera.read(). Si la lecture échoue, le programme affiche une erreur et s'arrête.

2. Reconnaissance toutes les deux secondes - Le test time.time() - dernier_test > 2 évite d'exécuter DeepFace sur chaque image. Cela rend l'affichage de la caméra plus fluide.

3. Recherche avec DeepFace - DeepFace.find compare l'image actuelle avec les photos du dossier. Le modèle VGG-Face est utilisé et OpenCV sert de détecteur de visage.

4. Récupération du nom - Lorsqu'une image correspondante est trouvée, le programme lit la colonne identity, récupère le nom du fichier avec os.path.basename, puis enlève son extension.

5. Affichage et fermeture - cv2.putText écrit le nom en vert sur l'image. cv2.imshow affiche la fenêtre. La touche Q arrête la boucle, puis release et destroyAllWindows ferment la caméra.

Partie centrale du code

```
if time.time() - dernier_test > 2:
    dernier_test = time.time()
    try:
        resultats = DeepFace.find(
            img_path=image,
            db_path=dossier,
            model_name="VGG-Face",
            detector_backend="opencv",
            enforce_detection=False,
            silent=True
        )
        if len(resultats) > 0 and resultats[0].empty == False:
            chemin_photo = resultats[0].iloc[0]["identity"]
            nom = os.path.splitext(os.path.basename(chemin_photo))[0]
        else:
            nom = "Inconnu"
    except:
        nom = "Aucun visage"
```

4. Résultats, limites et améliorations

Résultats obtenus

Le programme s'exécute correctement dans Jupyter Notebook. La webcam s'ouvre dans une nouvelle fenêtre et le nom de la personne est affiché lorsque son visage correspond à une image enregistrée. Lorsque DeepFace ne trouve pas de correspondance, le programme affiche « Inconnu ». Lorsque aucun visage exploitable n'est détecté ou qu'une exception se produit, il affiche « Aucun visage ».

Limites observées

- La base contient très peu de photos et ne représente pas toutes les variations du visage.
- La reconnaissance dépend de la lumière, de la distance et de l'orientation de la tête.
- Le bloc except ne montre pas le détail de l'erreur, ce qui rend le débogage plus difficile.
- Le système n'a pas encore été évalué avec un vrai jeu de test séparé.
- Les photos faciales sont des données personnelles et ne doivent pas être publiées sans autorisation.

Améliorations possibles

Pour améliorer le projet, il serait utile d'ajouter plusieurs photos par personne avec différentes expressions, positions et conditions d'éclairage. On pourrait également afficher un rectangle autour du visage, enregistrer la date et l'heure des reconnaissances, créer une interface graphique et remplacer le except général par un message d'erreur détaillé.

Conclusion

Ce projet nous a permis de comprendre les étapes essentielles d'une application de reconnaissance faciale : collecte des données, ouverture de la caméra, détection du visage, comparaison avec une base locale et affichage du résultat. Le code reste volontairement simple et accessible à un débutant. Il fonctionne comme démonstration, mais une utilisation réelle nécessiterait une base plus grande, des tests précis et une meilleure protection des données.

Fin du rapport